



METODOLOGIA DE  
ORDENAÇÃO INCREMENTAL

---

# Shell Sort e Comb Sort

**ALGORITMO E ESTRUTURA DE DADOS I**

Alunos: Gabriel Alves Faria, Rafael Alves Faria, Anthony Gabriel  
Sotto Mayor Silva e Henrique Parreira Cavalcante de Souza  
Orientador: Dr. Michel Pires da Silva

# Índice

---

- 01** INTRODUÇÃO
- 02** FUNDAMENTAÇÃO TEÓRICA
- 03** METODOLOGIA
- 04** MODELAGEM EXPERIMENTAL
- 05** RESULTADOS E DISCUSSÃO
- 06** CONCLUSÃO
- 07** REFERÊNCIAS

# Introdução

Algoritmos de ordenação são essenciais em sistemas que lidam com grandes volumes de dados. Mais do que funcionar corretamente, precisam escalar bem e se comportar de forma previsível.

Este trabalho analisa experimentalmente esses dois algoritmos, avaliando como tamanho da entrada, ordenação prévia dos dados e escolhas de implementação influenciam o desempenho na prática.



# Fundamentação Teórica

---

Os algoritmos de ordenação são um conjunto de instruções que recebem um array ou lista como entrada e organizam os itens em uma ordem específica.

São importantes para Ciência da Computação por reduzir a complexidade de um problema, resolvendo de forma eficiente e eficaz.



# Benefícios da ordenação

---

- **Pesquisa:** A pesquisa de um item em uma lista funciona muito mais rápido se a lista estiver ordenada.
- **Seleção:** Selecionar itens de uma lista com base em sua relação com os demais itens é mais fácil com dados ordenados.
- **Duplicados:** Encontrar valores duplicados em uma lista pode ser feito muito rapidamente quando a lista está ordenada.
- **Distribuição:** Analisar a distribuição de frequência dos itens em uma lista é muito rápido se a lista estiver ordenada.





# Fundamentação Teórica - Shell Sort

---

## Shell Sort — Conceitos

- Proposto por Donald Shell (1959);
- Refinamento do Insertion Sort com gap variável;
- Divide o array em sublistas intercaladas e ordena cada uma por inserção;
- Propriedade-chave: array h-ordenado permanece h-ordenado nas fases seguintes;
- Na última passagem, os elementos já estão quase ordenados.



# Fundamentação Teórica - Shell Sort

---

- Divide o array em sublistas intercaladas definidas por um intervalo de salto (gap);
- Aplica ordenação por inserção em cada sublista, permitindo que elementos se movam grandes distâncias de uma vez;
- Reduz o gap progressivamente a cada fase, usando uma sequência de incrementos (ex: Knuth: 1, 4, 13, 40...);



# Fundamentação Teórica - Shell Sort

---

- Quando o gap chega a 1, o array já está quase ordenado, tornando a passagem final muito barata;
- É instável — trocas entre elementos distantes podem alterar a ordem relativa de elementos iguais.





# Fundamentação Teórica

## - Shell Sort - Gaps

| Sequência       | Incrementos           | Complexidade (pior caso) | Autor                  |
|-----------------|-----------------------|--------------------------|------------------------|
| Shell           | 1, 2, ..., $N/2$      | $O(n^2)$                 | Shell (1959)           |
| Frank & Lazarus | 1, 3, ..., $2[N/4]+1$ | $O(n^{\{3/2\}})$         | Frank & Lazarus (1960) |
| Hibbard         | 1, 3, 7, 15, 31...    | $O(n^{\{3/2\}})$         | Hibbard (1963)         |
| Knuth           | 1, 4, 13, 40, 121...  | $O(n^{\{3/2\}})$         | Knuth (1973)           |
| Ciura           | 1, 4, 10, 23, 57...   | Empírico                 | Ciura (2001)           |

# Fundamentação Teórica - Shell Sort

---

Indicado quando elementos próximos estão muito distantes no array, tornando o Insertion Sort lento.

- Mais rápido que Bubble Sort e Insertion Sort simples;
- Eficiente para conjuntos de dados de tamanho médio;
- Reduz o número de trocas movendo elementos distantes rapidamente;
- Complexidade de tempo depende da sequência de gap utilizada;



# Fundamentação Teórica - Shell Sort

---

- Shell Sort — Complexidade (Sequência de Knuth)

| Caso   | Tempo         |
|--------|---------------|
| Melhor | $O(n \log n)$ |
| Médio  | $O(n^{3/2})$  |
| Pior   | $O(n^{3/2})$  |
| Espaço | $O(1)$        |

# Fundamentação Teórica - Comb Sort

---

Evolução do Bubble Sort que, assim como o Shell Sort faz com o Insertion Sort, introduz a comparação de elementos distantes antes dos adjacentes.

- É um algoritmo de ordenação por comparação;
- Usa pré-processamento para reduzir o gap entre comparações progressivamente;
- A estratégia é semelhante ao Shell Sort, mas aplicada sobre a lógica do Bubble Sort;



# Fundamentação Teórica - Comb Sort

---

- Começa comparando elementos separados por um gap grande proporcional ao array;
- A cada rodada, o gap é reduzido por um fator (geralmente 1.3);
- Quando o gap chega a 1, o algoritmo se comporta como um Bubble Sort convencional;
- Pares adjacentes fora de ordem são trocados até que o array esteja ordenado.





# Fundamentação Teórica - Comb Sort

---

| Shrink factor           | Pior caso | Referência          |
|-------------------------|-----------|---------------------|
| $\approx 1,33$          | $O(n^2)$  | Dobosiewicz (1980)  |
| 1,3                     | $O(n^2)$  | Lacey e Box (1991)  |
| $\sqrt{2} \approx 1,41$ | $O(n^2)$  | Análise comparativa |
| $\varphi \approx 1,618$ | $O(n^2)$  | Análise empírica    |

# Fundamentação Teórica - Comb Sort

---

Indicado como alternativa mais eficiente ao Bubble Sort puro, especialmente quando os dados estão desordenados de forma espalhada.

- Significativamente mais rápido que o Bubble Sort simples;
- Elimina "turtles" — valores pequenos no final do array que atrasam o Bubble Sort;
- Simples de implementar, com ganho de desempenho relevante;
- A escolha do fator de redução do gap impacta diretamente a eficiência.



# Fundamentação Teórica - Comb Sort

---

- Comb Sort — Complexidade (shrink factor = 1,3)

| Caso   | Tempo                              |
|--------|------------------------------------|
| Melhor | $O(n \log n)$                      |
| Médio  | $O(n^2 / 2^p) \approx O(n \log n)$ |
| Pior   | $O(n^2)$                           |
| Espaço | $O(1)$                             |



# Metodologia

---

## SHELL SORT

Ideia central Insertion Sort com gap variável → elementos se movem distâncias maiores por vez.

Sequência de Knuth:  $h = h \times 3 + 1$  (começa em 1, cresce até  $< n$ ).

Passos:

1. Calcular o maior gap inicial;
2. Ordenar sublistas intercaladas com esse gap;
3. Reduzir o gap e repetir;
4. Encerrar após varredura final com gap = 1;

## Algoritmo: ShellSort(vetor)

```
1:  $n \leftarrow \text{tamanho}(\text{vetor})$ 
2:  $gap \leftarrow 1$ 
3: enquanto  $gap * 3 + 1 < n$  faça
4:    $gap \leftarrow gap * 3 + 1$ 
5: fim_enquanto
6: enquanto  $gap > 0$  faça
7:    $i \leftarrow gap$ 
8:   enquanto  $i < n$  faça
9:      $temp \leftarrow \text{vetor}[i]$ 
10:     $j \leftarrow i - gap$ 
11:    enquanto  $j \geq 0$  e  $\text{vetor}[j] > temp$  faça
12:       $\text{vetor}[j + gap] \leftarrow \text{vetor}[j]$ 
13:       $j \leftarrow j - gap$ 
14:    fim_enquanto
15:     $\text{vetor}[j + gap] \leftarrow temp$ 
16:     $i \leftarrow i + 1$ 
17:  fim_enquanto
18:   $gap \leftarrow \lfloor gap / 3 \rfloor$ 
19: fim_enquanto
```

# Metodologia

# Metodologia

---

## COMB SORT

Ideia central Bubble Sort com gap variável → elimina "turtles".

Fator de encolhimento: 1,3:  $\text{gap} = (\text{gap} \times 10) / 13$

Passos:

1. Iniciar com gap = tamanho do array;
2. Comparar e trocar pares separados pelo gap;
3. Reduzir gap a cada ciclo;
4. Encerrar quando gap = 1 sem nenhuma troca.



## Algoritmo: CombSort(vetor)

```
1:  gap ← tamanho(vetor)
2:  shrink ← 1,3
3:  ordenado ← FALSO
4:  enquanto não ordenado faça
5:      gap ← ⌊ gap / shrink ⌋
6:      se gap < 1 então
7:          gap ← 1
8:          ordenado ← VERDADEIRO
9:      fim_se
10:     i ← 0
11:     enquanto i + gap < tamanho(vetor) faça
12:         se vetor[i] > vetor[i + gap] então
13:             vetor[i] ↔ vetor[i + gap]
14:             ordenado ← FALSO
15:         fim_se
16:         i ← i + 1
17:     fim_enquanto
18: fim_enquanto
```

# Metodologia

# Metodologia

---

## IMPLEMENTAÇÃO

- 5 linguagens: Python · C · C++ · Java · Go.

Padronizado entre todas:

- Mesma lógica e sequência de gaps;
- Tempo medido só na rotina de ordenação.

# Metodologia

---

## IMPLEMENTAÇÃO

Diferenças esperadas por linguagem;

- Python → mais lento (interpretado);
- C / C++ → mais rápido (compilado, acesso direto à memória);
- Java → JVM com JIT;
- Go → compilado, execução enxuta.

# Modelagem Experimental

---

## AMBIENTE COMPUTACIONAL

- Editores: VS Code e IntelliJ IDEA;
- Processador: Intel U300 (1,20 GHz) · 8 GB RAM;
- Sistema: Ubuntu 24.04 LTS via WSL;
- Ferramentas: GCC 13.3 · OpenJDK 21 · Python 3.12 · Go 1.26.3

# Modelagem Experimental

## AMBIENTE COMPUTACIONAL

| Linguagem | Modelo de execução                                                   |
|-----------|----------------------------------------------------------------------|
| C/C++     | Compilada para código nativo, acesso direto à memória                |
| Java      | Compilada para bytecode, executada pela JVM com JIT                  |
| Go        | Compilada estaticamente para código nativo, com garbage collector    |
| Python    | Compilada para bytecode, interpretada pelo CPython, tipagem dinâmica |



# Modelagem Experimental

---

## DEFINIÇÃO DOS EXPERIMENTOS

- 3 tipos de entrada: aleatória, crescente e decrescente
- Inteiros distintos no intervalo  $[1, N]$ ;
- 5 tamanhos:  $10^2$ ,  $10^3$ ,  $10^4$ ,  $10^5$ ,  $10^6$ ;
- Objetivo: avaliar a sensibilidade dos algoritmos à ordenação prévia dos dados.

# Resultados e Discussão

---

## ANÁLISE

- C foi a linguagem mais rápida em quase todos os cenários.
- C++ teve desempenho muito próximo, com pequeno overhead de abstração.
- Go surpreendeu positivamente, competindo diretamente com C/C++.
- Java apresentou bom desempenho em entradas grandes devido ao JIT Compiler.
- Python teve os maiores tempos por causa do overhead do interpretador CPython.

# Resultados e Discussão

---

## ALGORITMOS

- Shell Sort foi superior em dados ordenados ou parcialmente ordenados.
- Comb Sort foi ligeiramente melhor em entradas aleatórias massivas.
- Em C, C++, Java e Go, o Comb Sort superou o Shell Sort no cenário aleatório com  $n=10^6$ .
- Em Python, o Shell Sort foi melhor devido ao custo extra das passagens do Comb Sort.

# Resultados e Discussão



---

## ALGORITMOS

- A complexidade assintótica dos algoritmos foi confirmada em todas as linguagens.
- O principal fator de diferença foi o custo por operação imposto pelo ambiente de execução.
- C apresentou o melhor desempenho geral, graças à compilação direta para código de máquina, baixo overhead e acesso eficiente à memória.

# Resultados e Discussão



---

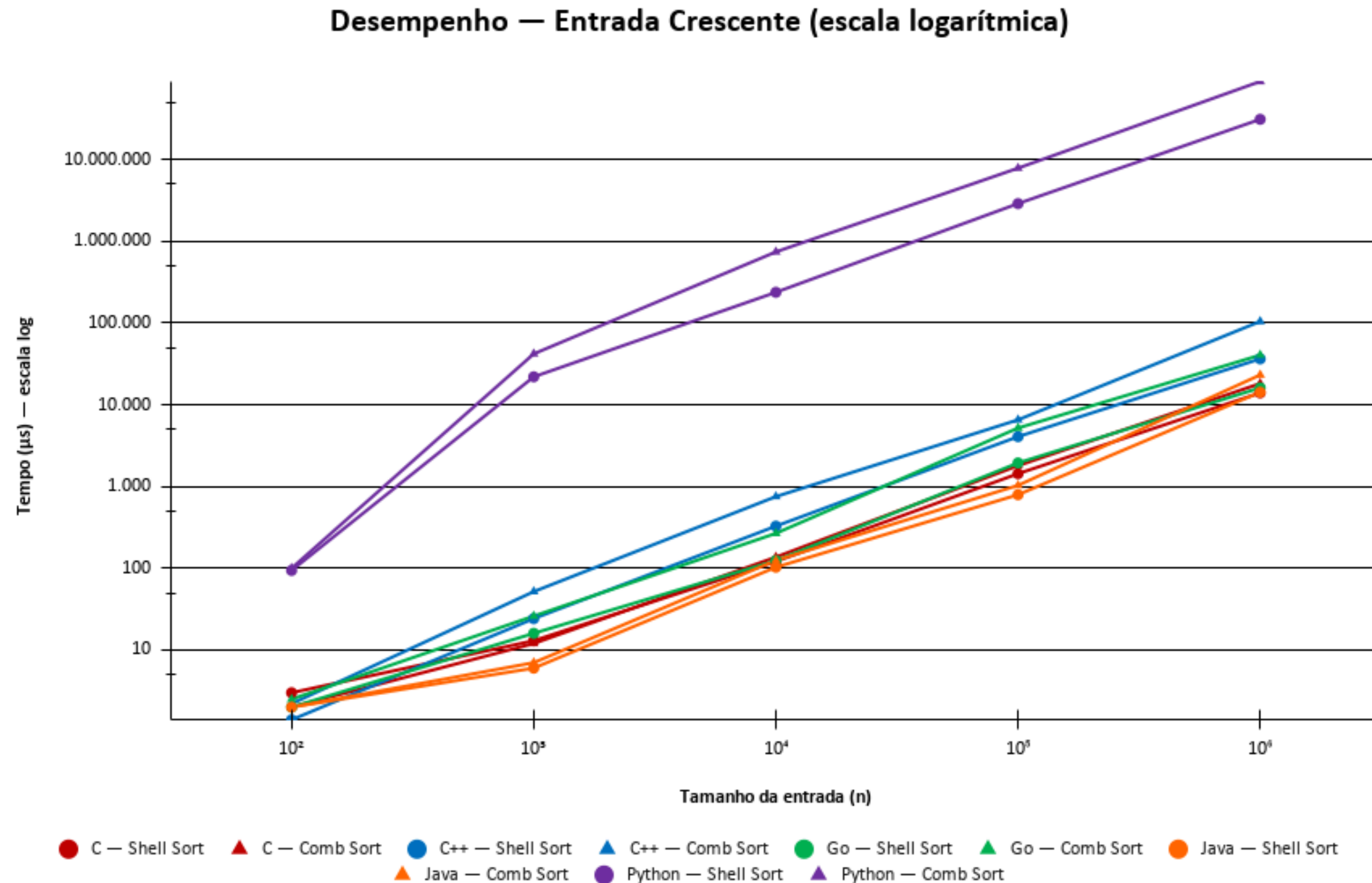
## ALGORITMOS

- Go e C++ mostraram excelente desempenho;
- Já o Java equilibrou velocidade, portabilidade e desempenho competitivo em entradas grandes, graças ao JIT Compiler;
- Python, embora produtivo, não é ideal para algoritmos intensivos em comparações e laços repetitivos.



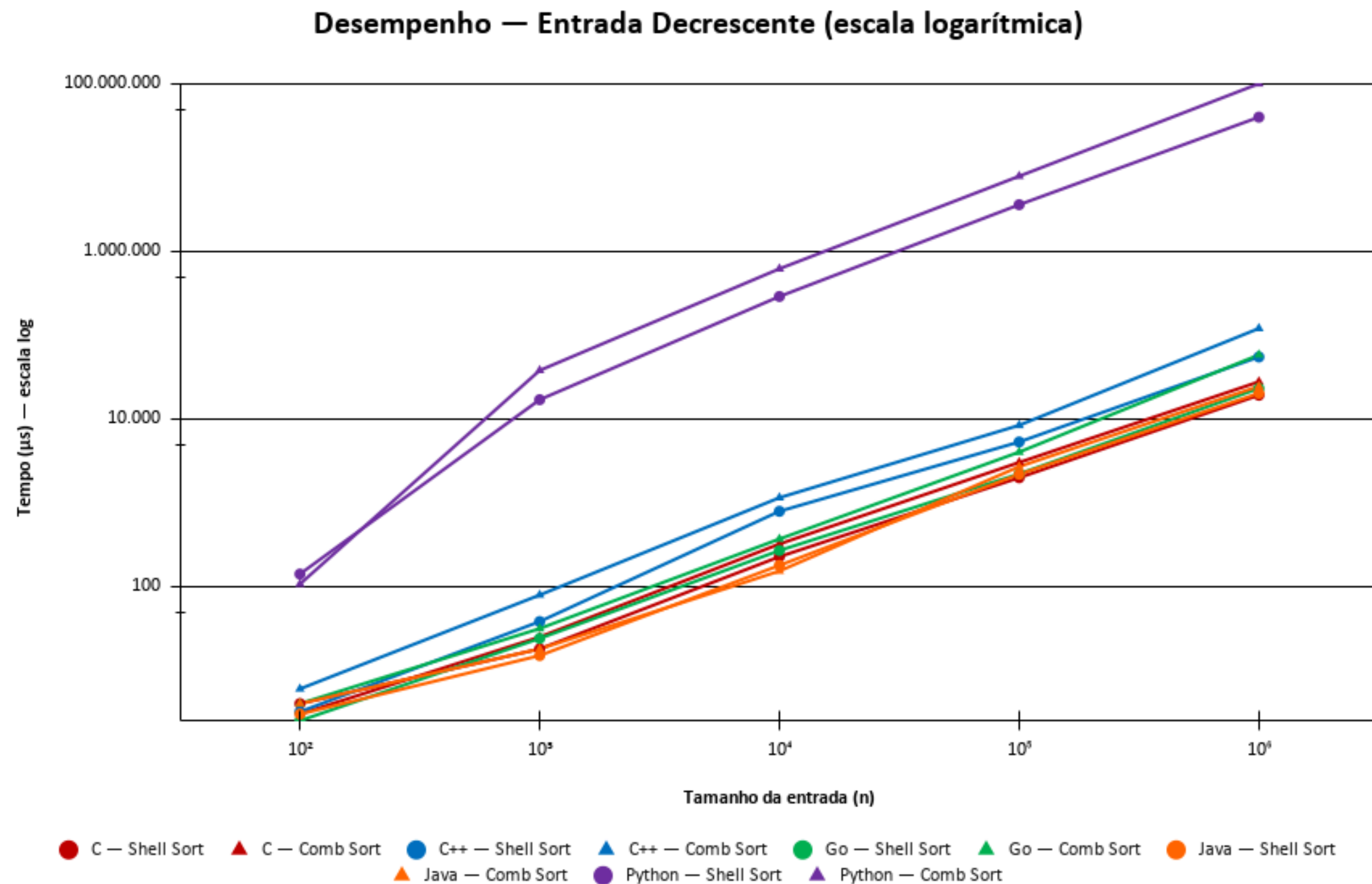
# Resultados e Discussão

## GRÁFICOS



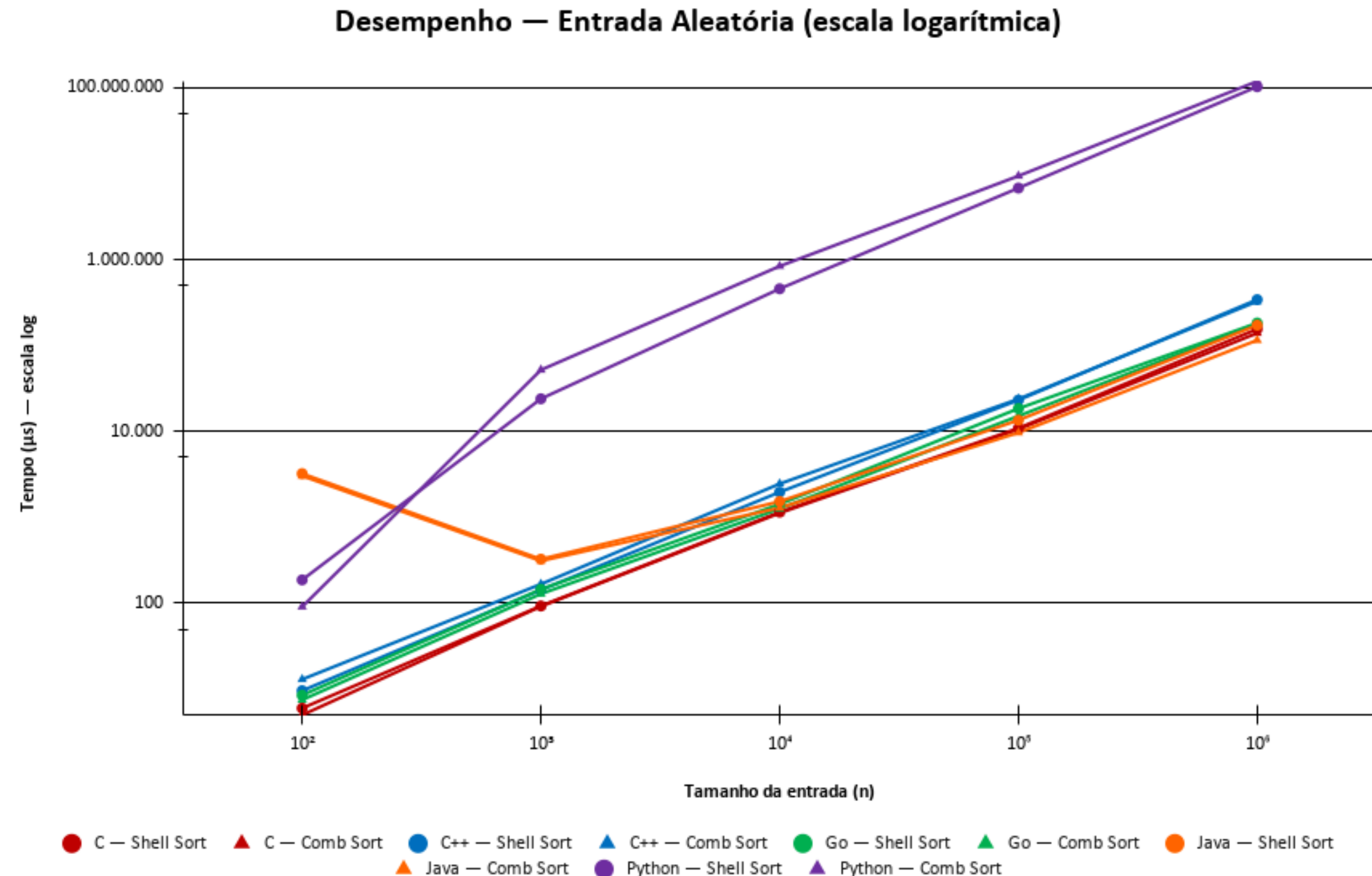
# Resultados e Discussão

## GRÁFICOS



# Resultados e Discussão

## GRÁFICOS





# Conclusão

---

- O desempenho do Shell Sort e do Comb Sort depende da distribuição dos dados e do ambiente de execução;
- Shell Sort foi mais eficiente em dados parcialmente ordenados; Comb Sort destacou-se em dados aleatórios massivos;
- C apresentou os melhores tempos de execução, seguido de C++ e Go, mantendo baixo consumo de memória;
- Java teve bom desempenho graças ao JIT Compiler, mas com alto uso de RAM.
- Python apresentou os maiores tempos devido ao overhead do interpretador.

# Conclusão

- Limitações: restrição a uma única sequência de gaps (Knuth) e fator fixo de 1,3; ambiente virtualizado (WSL) com processador híbrido sujeito a variações térmicas; ausência de análise estatística formal (intervalo de confiança e desvio padrão); e heterogeneidade dos métodos de medição de memória entre linguagens.
- Trabalhos futuros: testar novas sequências/fatores e avaliar desempenho com estruturas de dados complexas.

# Referências

---

- SHELL, D. L. A High-Speed Sorting Procedure. Communications of the ACM, 1959.
- LACEY, S.; BOX, R. A Fast, Easy Sort. Byte Magazine, 1991.
- KNUTH, D. E. The Art of Computer Programming: Sorting and Searching. Addison-Wesley, 1973.
- HIBBARD, T. N. An Empirical Study of Minimal Storage Sorting. CACM, 1963.
- CIURA, M. Best Increments for the Average Case of Shellsort. Springer, 2001.
- ZIVIANI, N. Projeto de Algoritmos. 2. ed. São Paulo: Pioneira Thomson Learning, 2004.